Phase 3 — Python Forecasting + Correlation

Tasks:

1. Load & prep monthly time series from MySQL
2. Prophet forecast: retail spend next 12 months (BOE rate as regressor)
3. Pearson correlation: BOE rate rises vs closure spikes by sector

```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine
from prophet import Prophet
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import warnings
warnings.filterwarnings('ignore')

try:
    from scipy.stats import pearsonr
except ModuleNotFoundError:
    def pearsonr(x, y):
        x = np.asarray(x, dtype=float)
        y = np.asarray(y, dtype=float)
        r = np.corrcoef(x, y)[0, 1]
        return r, np.nan

engine =
create_engine('mysql+mysqlconnector://root:******@127.0.0.1/uk_high_st
reet')
print('✓ Ready')
```

```
✓ Ready
```

```
# ── FIX: Rebuild company_closures from raw_dissolutions
──────────────────────

import pandas as pd

# Pull retail SIC47 closures from raw table
raw = pd.read_sql("""
    SELECT
        CompanyName,
        `SICCode.SicText_1` AS sic_text,
        `RegAddress.County` AS county,
        CompanyStatus,
        DissolutionDate,
        IncorporationDate
    FROM raw_dissolutions
    WHERE CompanyStatus IN (
```

```
        'Active – Proposal to Strike off','Liquidation','In
Administration',
        'Live but Receiver Manager on at least one charge',
        'In Administration/Administrative Receiver','ADMINISTRATION
ORDER',
        'Voluntary Arrangement','VOLUNTARY ARRANGEMENT / RECEIVER
MANAGER',
        'In Administration/Receiver Manager'
    )
    AND (
        `SICCode.SicText_1` LIKE '47%' OR `SICCode.SicText_2` LIKE
'47%'
        OR `SICCode.SicText_3` LIKE '47%' OR `SICCode.SicText_4` LIKE
'47%'
    )
""", engine)

print(f"Rows pulled: {len(raw)}")
print(f"DissolutionDate sample:
{raw['DissolutionDate'].head(5).tolist()}")
print(f"DissolutionDate dtype: {raw['DissolutionDate'].dtype}")
```

```
Rows pulled: 6553
DissolutionDate sample: [None, None, None, None, None]
DissolutionDate dtype: object
```

Cell 1 — Load & Prep Monthly Time Series

```
# — 1a. ONS retail spend
————————————————————————————————————————————————————————————

# Use 'All Retailing' / UK-level aggregate for forecasting
ons_raw = pd.read_sql("""
    SELECT time_period, geography, retail_category, value
    FROM ons_retail_sales
    ORDER BY time_period
""", engine, parse_dates=['time_period'])

print('ONS shape:', ons_raw.shape)
print('Categories:', ons_raw['retail_category'].unique()[:10])
print('Geographies:', ons_raw['geography'].unique()[:5])
```

```
ONS shape: (29778, 4)
Categories: ['textile-clothing-footwear-and-leather-all-businesses'
 'medical-goods-all-businesses' 'dispensing-chemists-all-businesses'
 'sports-equipment-games-and-toys-all-businesses'
 'watches-and-jewellery-all-businesses' 'second-hand-goods-all-
businesses'
 'mail-order-all-businesses'
```

```
 'alcoholic-drinks-other-beverages-and-tobacco-all-businesses'
 'clothing-all-businesses' 'furniture-lighting-etc-all-businesses']
Geographies: ['Great Britain']
```

```python
# ── 1b. Filter to UK aggregate + broadest retail category ──────────────────────
# Adjust category/geography strings if your ONS data uses different labels
UK_GEO   = 'K03000001'   # ONS code for United Kingdom — change if needed
ALL_CAT  = 'all-retailing-including-automotive-fuel'  # change to match your data

spend = ons_raw[
    (ons_raw['geography'] == UK_GEO) &
    (ons_raw['retail_category'] == ALL_CAT)
][['time_period', 'value']].copy()

# Fallback: if filter returns nothing, use first available geography + category
if spend.empty:
    first_geo = ons_raw['geography'].iloc[0]
    first_cat = ons_raw['retail_category'].iloc[0]
    print(f'⚠ Fallback → geography={first_geo}, category={first_cat}')
    spend = ons_raw[
        (ons_raw['geography'] == first_geo) &
        (ons_raw['retail_category'] == first_cat)
    ][['time_period', 'value']].copy()

spend = spend.sort_values('time_period').reset_index(drop=True)
spend.columns = ['ds', 'y']
spend['ds'] = pd.to_datetime(spend['ds'])

print('Spend rows:', len(spend))
print(spend.head(3))
print(spend.tail(3))
```

```
⚠ Fallback → geography=Great Britain, category=textile-clothing-footwear-and-leather-all-businesses
Spend rows: 914
          ds     y
0 1988-01-01  36.9
1 1988-01-01  32.0
2 1988-02-01  36.2
            ds      y
911 2025-12-01  103.2
912 2026-01-01  100.0
913 2026-01-01  103.3
```

```python
# ── 1c. BOE base rate → monthly forward-fill ──────────────────────────────
boe_raw = pd.read_sql("""
    SELECT date_changed, rate
    FROM boe_base_rate
    ORDER BY date_changed
""", engine, parse_dates=['date_changed'])

# Build a full monthly series from BOE first change to last ONS month
date_range = pd.date_range(
    start=boe_raw['date_changed'].min(),
    end=spend['ds'].max(),
    freq='MS'  # month start
)

boe_monthly = (
    boe_raw
    .set_index('date_changed')
    .reindex(date_range)
    .ffill()              # forward-fill: rate stays until next change
    .reset_index()
    .rename(columns={'index': 'ds', 'rate': 'boe_rate'})
)

print('BOE monthly rows:', len(boe_monthly))
print(boe_monthly.tail(5))
```

```
BOE monthly rows: 600
            ds  boe_rate
595 2025-09-01       5.0
596 2025-10-01       5.0
597 2025-11-01       5.0
598 2025-12-01       5.0
599 2026-01-01       5.0
```

```python
# ── 1d. Company closures → monthly count (robust + hard validation) ──────
closures_raw = pd.read_sql("""
    SELECT company_name, dissolution_date, incorporation_date,
sic_text, county, postcode
    FROM company_closures
""", engine)

# Parse dissolution_date from curated table first
closures_raw['dissolution_date'] = pd.to_datetime(
    closures_raw['dissolution_date'], errors='coerce', dayfirst=True
)
valid_dates = closures_raw['dissolution_date'].notna().sum()
print("company_closures valid dissolution_date:", valid_dates)
```

```
# Fallback: if curated table has no usable dates, pull directly from
raw_dissolutions
if valid_dates == 0:
    print("⚠ Falling back to raw_dissolutions (company_closures has no
valid dissolution dates)")
    closures_raw = pd.read_sql("""
        SELECT
            CompanyName AS company_name,
            COALESCE(
                CASE WHEN `SICCode.SicText_1` LIKE '47%' THEN
`SICCode.SicText_1` END,
                CASE WHEN `SICCode.SicText_2` LIKE '47%' THEN
`SICCode.SicText_2` END,
                CASE WHEN `SICCode.SicText_3` LIKE '47%' THEN
`SICCode.SicText_3` END,
                CASE WHEN `SICCode.SicText_4` LIKE '47%' THEN
`SICCode.SicText_4` END
            ) AS sic_text,
            `RegAddress.County` AS county,
            `RegAddress.PostCode` AS postcode,
            DissolutionDate AS dissolution_date,
            IncorporationDate AS incorporation_date
        FROM raw_dissolutions
        WHERE (
              `SICCode.SicText_1` LIKE '47%' OR `SICCode.SicText_2`
LIKE '47%'
              OR `SICCode.SicText_3` LIKE '47%' OR `SICCode.SicText_4`
LIKE '47%'
          )
    """, engine)

    closures_raw['dissolution_date'] = pd.to_datetime(
        closures_raw['dissolution_date'], errors='coerce',
dayfirst=True
    )

# Hard stop: avoid silent empty downstream outputs
valid_dates_after = closures_raw['dissolution_date'].notna().sum()
if valid_dates_after == 0:
    raise ValueError(
        "No usable dissolution dates found in
company_closures/raw_dissolutions. "
        "Fix Phase 1 ingestion and source CSV first, then rerun from
Cell 8."
    )

# Keep only rows with valid date + sector
closures_raw = closures_raw.dropna(subset=['dissolution_date',
'sic_text']).copy()
print("After parse — non-null dissolution_date:",
```

```
closures_raw['dissolution_date'].notna().sum())
print("Sample:\n", closures_raw.head(3))

# Monthly count by sector
closures_raw['month'] =
closures_raw['dissolution_date'].dt.to_period('M').dt.to_timestamp()

closures_by_sector = (
    closures_raw
    .groupby(['month', 'sic_text'])
    .size()
    .reset_index(name='closure_count')
)

# Total monthly (for master frame)
closures_monthly = (
    closures_by_sector
    .groupby('month')['closure_count']
    .sum()
    .reset_index()
    .rename(columns={'month': 'ds', 'closure_count':
'total_closures'})
)

print(f'\nclosures_by_sector rows: {len(closures_by_sector)}')
print(f'closures_monthly rows: {len(closures_monthly)}')
print(closures_monthly.head(3))
```

```
company_closures valid dissolution_date: 54538
After parse — non-null dissolution_date: 54538
Sample:
           company_name dissolution_date incorporation_date  \
0  $$ DEALS $$ LIMITED       2021-11-20         2013-11-20
1         $APPAREL LTD       2025-12-12         2024-11-29
2             &EVE LTD       2026-01-23         2025-01-10

                                            sic_text  county  postcode

0  47910 - Retail sale via mail order houses or v...    None  SW19 2RR

1  47710 - Retail sale of clothing in specialised...  SURREY   CR7 8BB

2  47710 - Retail sale of clothing in specialised...    None   YO8 4DB


closures_by_sector rows: 3668
closures_monthly rows: 191
          ds  total_closures
0 2008-12-01               1
```

```
1 2010-09-01                1
2 2010-10-01                2
```

```
# ── 1e. Master monthly frame ──────────────────────────────────────────────
master = (
    spend
    .merge(boe_monthly[['ds', 'boe_rate']], on='ds', how='left')
    .merge(closures_monthly, on='ds', how='left')
)

master['boe_rate']        = master['boe_rate'].ffill()
master['total_closures']  = master['total_closures'].fillna(0)

print('Master shape:', master.shape)
print(master.tail(5))
print('\nNull check:')
print(master.isnull().sum())
```

```
Master shape: (914, 4)
            ds      y  boe_rate  total_closures
909 2025-11-01  104.3       5.0          3170.0
910 2025-12-01   99.9       5.0          3787.0
911 2025-12-01  103.2       5.0          3787.0
912 2026-01-01  100.0       5.0           955.0
913 2026-01-01  103.3       5.0           955.0

Null check:
ds                 0
y                  0
boe_rate           0
total_closures     0
dtype: int64
```

Cell 2 — Prophet Forecast: Retail Spend (12 months)

```
# ── 2a. Build Prophet model with BOE rate as extra regressor ──────────────
prophet_df = master[['ds', 'y', 'boe_rate']].dropna().copy()

m = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    daily_seasonality=False,
    changepoint_prior_scale=0.05,   # lower = smoother trend
    seasonality_prior_scale=10
)
m.add_regressor('boe_rate')
```

```
m.fit(prophet_df)

print('✓ Model trained on', len(prophet_df), 'months')
```

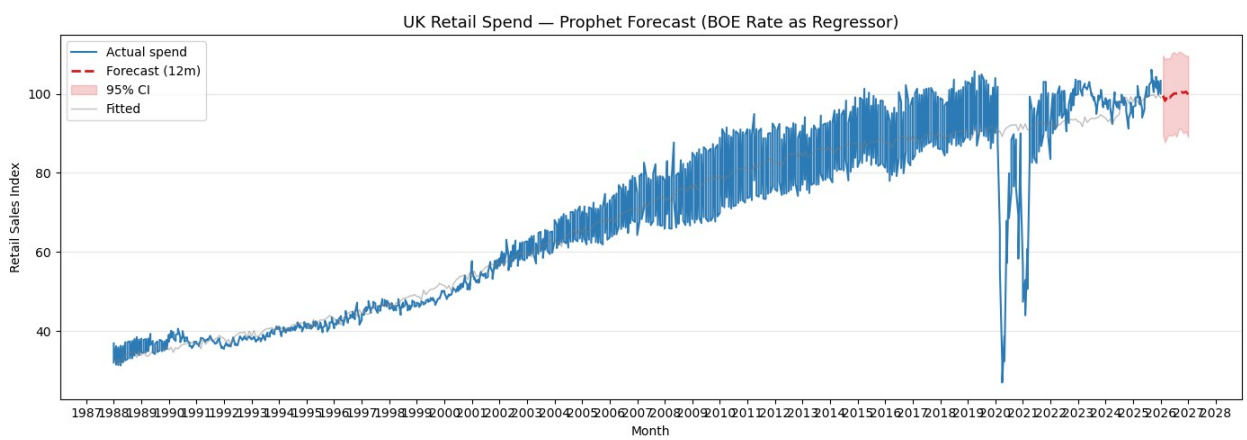
```
17:20:24 - cmdstanpy - INFO - Chain [1] start processing
17:20:24 - cmdstanpy - INFO - Chain [1] done processing

✓ Model trained on 914 months
```

```
# ── 2b. Build future dataframe (12 months) ──────────────────────────────────
future = m.make_future_dataframe(periods=12, freq='MS')

# Forward-fill BOE rate into future: assume last known rate holds
last_rate = prophet_df['boe_rate'].iloc[-1]
future = future.merge(boe_monthly[['ds', 'boe_rate']], on='ds',
how='left')
future['boe_rate'] = future['boe_rate'].ffill().fillna(last_rate)

forecast = m.predict(future)
print('✓ Forecast complete')
print(forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(14))
```

```
✓ Forecast complete
            ds        yhat  yhat_lower  yhat_upper
455 2025-12-01   99.902635   89.436483  109.479867
456 2026-01-01   98.953045   89.160835  109.130730
457 2026-02-01   99.524574   89.489590  109.733693
458 2026-03-01   98.242683   87.680185  108.645490
459 2026-04-01   99.393653   89.292867  109.054494
460 2026-05-01   99.024664   89.628314  108.874165
461 2026-06-01   99.653157   89.356432  110.307274
462 2026-07-01  100.096016   89.971728  110.513162
463 2026-08-01  100.100970   89.167292  109.944960
464 2026-09-01  100.172491   91.149960  110.666021
465 2026-10-01  100.406340   91.063023  110.482282
466 2026-11-01  100.298579   89.971075  110.009902
467 2026-12-01  100.519816   90.553260  109.669987
468 2027-01-01   99.894216   89.038397  109.584657
```

```
# ── 2c. Plot forecast ──────────────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(14, 5))

# Historical
ax.plot(prophet_df['ds'], prophet_df['y'],
        color='#2c7bb6', linewidth=1.5, label='Actual spend')

# Forecast
future_only = forecast[forecast['ds'] > prophet_df['ds'].max()]
ax.plot(future_only['ds'], future_only['yhat'],
```

```
        color='#d7191c', linewidth=2, linestyle='--', label='Forecast
(12m)')
ax.fill_between(future_only['ds'],
                future_only['yhat_lower'],
                future_only['yhat_upper'],
                alpha=0.2, color='#d7191c', label='95% CI')

# Fitted (in-sample)
in_sample = forecast[forecast['ds'] <= prophet_df['ds'].max()]
ax.plot(in_sample['ds'], in_sample['yhat'],
        color='grey', linewidth=1, alpha=0.5, label='Fitted')

ax.xaxis.set_major_locator(mdates.YearLocator())
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
ax.set_title('UK Retail Spend — Prophet Forecast (BOE Rate as
Regressor)', fontsize=13)
ax.set_ylabel('Retail Sales Index')
ax.set_xlabel('Month')
ax.legend()
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('phase3_forecast.png', dpi=150)
plt.show()
print('✓ Saved: phase3_forecast.png')
```




```
✓ Saved: phase3_forecast.png

# ── 2d. Export forecast to MySQL
──────────────────────────────────────────────────

forecast_export = forecast[['ds', 'yhat', 'yhat_lower',
'yhat_upper']].copy()
forecast_export.columns = ['forecast_month', 'yhat', 'yhat_lower',
'yhat_upper']

forecast_export.to_sql('retail_spend_forecast', engine,
```

```
                          if_exists='replace', index=False)
print(f'✓ retail_spend_forecast written: {len(forecast_export)} rows')
```

```
✓ retail_spend_forecast written: 469 rows
```

Cell 3 — Pearson Correlation: BOE Rate vs Closures by Sector

```
# ── 3a. Sector-level monthly closures (robust) ──────────────────────────────
print("closures_raw columns:", closures_raw.columns.tolist())
print("boe_monthly columns:", boe_monthly.columns.tolist())

# Build monthly sector counts if not already present
if 'closure_count' not in closures_raw.columns:
    sector_base = (
        closures_raw
        .dropna(subset=['month', 'sic_text'])
        .groupby(['month', 'sic_text'], as_index=False)
        .size()
        .rename(columns={'size': 'closure_count'})
    )
else:
    sector_base = closures_raw[['month', 'sic_text',
'closure_count']].dropna(subset=['month', 'sic_text'])

sector_closures = (
    sector_base
    .merge(boe_monthly[['ds', 'boe_rate']], left_on='month',
right_on='ds', how='inner')
    .dropna(subset=['boe_rate'])
)

print("\nsector_base shape:", sector_base.shape)
print("sector_closures shape:", sector_closures.shape)
print("sector_closures columns:", sector_closures.columns.tolist())

# Count unique months per sector
sector_counts = sector_closures.groupby('sic_text')['month'].nunique()
print("\nMonths per sector (top 10):\n",
sector_counts.sort_values(ascending=False).head(10))

MIN_MONTHS = 6
valid_sectors = sector_counts[sector_counts >= MIN_MONTHS].index
print(f'\nValid sectors (≥{MIN_MONTHS} months): {len(valid_sectors)}')
```

```
sector_closures =
sector_closures[sector_closures['sic_text'].isin(valid_sectors)]
```

```
closures_raw columns: ['company_name', 'dissolution_date',
'incorporation_date', 'sic_text', 'county', 'month']
boe_monthly columns: ['ds', 'boe_rate']

sector_base shape: (3668, 3)
sector_closures shape: (3295, 5)
sector_closures columns: ['month', 'sic_text', 'closure_count', 'ds',
'boe_rate']

Months per sector (top 10):
 sic_text
47110 - Retail sale in non-specialised stores with food, beverages or
tobacco predominating                    121
47190 - Other retail sale in non-specialised stores
117
47710 - Retail sale of clothing in specialised stores
111
47910 - Retail sale via mail order houses or via Internet
108
47990 - Other retail sale not in stores, stalls or markets
107
47789 - Other retail sale of new goods in specialised stores (not
commercial art galleries and opticians)      101
47290 - Other retail sale of food in specialised stores
100
47599 - Retail of furniture, lighting, and similar (not musical
instruments or scores) in specialised store    100
47250 - Retail sale of beverages in specialised stores
91
47530 - Retail sale of carpets, rugs, wall and floor coverings in
specialised stores                             88
Name: month, dtype: int64

Valid sectors (≥6 months): 44
```

```
results = []
skipped_flat = 0

for sector, grp in sector_closures.groupby('sic_text'):
    grp = grp.sort_values('month')
    if grp['closure_count'].std() == 0:
        skipped_flat += 1
        continue  # skip flat series

    r, p = pearsonr(grp['boe_rate'], grp['closure_count'])
    results.append({
        'sector': sector,
```

```
        'pearson_r': round(r, 4),
        'p_value': round(p, 4) if not np.isnan(p) else None,
        'months': len(grp),
        'mean_closures': round(grp['closure_count'].mean(), 1)
    })

corr_df = pd.DataFrame(results)

if corr_df.empty:
    print('No valid sectors available for Pearson correlation after
filtering.')
    print(f"Sectors checked: {sector_closures['sic_text'].nunique()},
flat/constant skipped: {skipped_flat}")
    corr_df = pd.DataFrame(columns=['sector', 'pearson_r', 'p_value',
'months', 'mean_closures'])
else:
    corr_df = corr_df.sort_values('pearson_r', ascending=False)
    print(corr_df.to_string(index=False))
```

```
sector  pearson_r p_value  months  mean_closures
                                                 47300 - Retail sale of
automotive fuel in specialised stores    -0.3735    None      54
1.9

47741 - Retail sale of hearing aids    -0.4145    None      24
1.5
                                                      47591 - Retail
sale of musical instruments and scores    -0.4483    None      43
2.0
                                       47430 - Retail sale of audio and
video equipment in specialised stores    -0.4522    None      54
2.3
                                                 47722 - Retail sale
of leather goods in specialised stores    -0.4538    None      43
2.5
                                       47620 - Retail sale of newspapers
and stationery in specialised stores    -0.4583    None      73
2.8
                                      47630 - Retail sale of music and
video recordings in specialised stores    -0.4811    None      45
2.5
                                          47210 - Retail sale of fruit
and vegetables in specialised stores    -0.4943    None      80
5.6
                                47799 - Retail sale of other second-hand
goods in stores (not incl. antiques)    -0.5121    None      66
3.8
                                 47540 - Retail sale of electrical
household appliances in specialised stores    -0.5127    None      73
```

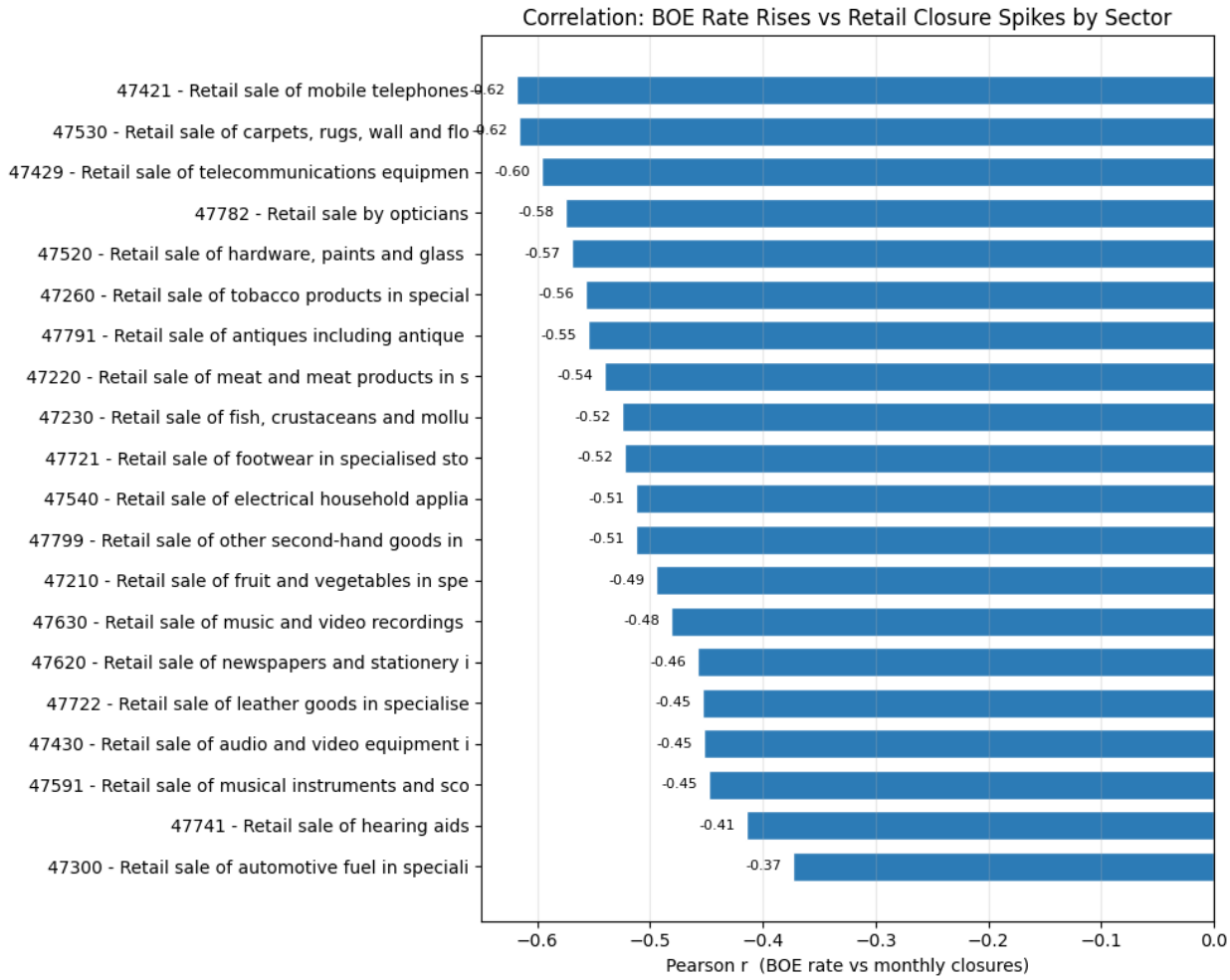
6.6
47721 - Retail sale of footwear in specialised stores -0.5229 None 71 3.3
47230 - Retail sale of fish, crustaceans and molluscs in specialised stores -0.5248 None 54 2.5
47220 - Retail sale of meat and meat products in specialised stores -0.5402 None 83 5.9
47791 - Retail sale of antiques including antique books in stores -0.5547 None 53 2.8
47260 - Retail sale of tobacco products in specialised stores -0.5576 None 74 4.2
47520 - Retail sale of hardware, paints and glass in specialised stores -0.5695 None 79 3.9

47782 - Retail sale by opticians -0.5753 None 54 2.3
47429 - Retail sale of telecommunications equipment other than mobile telephones -0.5957 None 68 3.4
47530 - Retail sale of carpets, rugs, wall and floor coverings in specialised stores -0.6162 None 88 6.7

47421 - Retail sale of mobile telephones -0.6185 None 82 6.9
47781 - Retail sale in commercial art galleries -0.6315 None 57 3.6
47610 - Retail sale of books in specialised stores -0.6385 None 51 3.2
47730 - Dispensing chemist in specialised stores -0.6508 None 54 5.0
47410 - Retail sale of computers, peripheral units and software in specialised stores -0.6562 None 81 6.8
47510 - Retail sale of textiles in specialised stores -0.6563 None 77 6.3
47820 - Retail sale via stalls and markets of textiles, clothing and footwear -0.6646 None 74 6.0

47650 - Retail sale of games and toys in specialised stores -0.6648 None 71 5.6
47990 - Other retail sale not in stores, stalls or markets -0.6689 None 107 47.1
47750 - Retail sale of cosmetic and toilet articles in specialised stores -0.6737 None 87 12.2
47599 - Retail of furniture, lighting, and similar (not musical instruments or scores) in specialised store -0.6787 None 100 13.7
47810 - Retail sale via stalls and markets of food, beverages and tobacco products -0.6816 None 66 3.9
47749 - Retail sale of medical and orthopaedic goods in specialised stores (not incl. hearing aids) n.e.c. -0.6921 None 61 3.6
47250 - Retail sale of beverages in specialised stores -0.7001 None 91 6.3
47789 - Other retail sale of new goods in specialised stores (not commercial art galleries and opticians) -0.7040 None 101 12.8
47290 - Other retail sale of food in specialised stores -0.7044 None 100 11.7
47760 - Retail sale of flowers, plants, seeds, fertilizers, pet animals and pet food in specialised stores -0.7060 None 77 9.5
47240 - Retail sale of bread, cakes, flour confectionery and sugar confectionery in specialised stores -0.7075 None 82 8.9
47640 - Retail sale of sports goods, fishing gear, camping goods, boats and bicycles -0.7298 None 80 11.5
47770 - Retail sale of watches and jewellery in specialised stores -0.7309 None 82 8.3
47910 - Retail sale via mail order houses or via Internet -0.7467 None 108 149.8
47890 - Retail sale via stalls and markets of other goods -0.7550 None 78 7.0
47110 - Retail sale in non-specialised stores with food, beverages or tobacco predominating -0.7577 None 121 34.3
47710 - Retail

```
sale of clothing in specialised stores    -0.7599    None     111
30.8
                                                                47190 - Other
retail sale in non-specialised stores    -0.7688    None     117
37.2

# ── 3c. Plot: correlation heatmap by sector
──────────────────────────────────────────
if corr_df.empty:
    print('Skipping plot: corr_df is empty (no valid sector
correlations to display).')
else:
    top_n = corr_df.head(20)  # top 20 sectors by r value

    colors = ['#d7191c' if r > 0 else '#2c7bb6' for r in
top_n['pearson_r']]

    fig, ax = plt.subplots(figsize=(10, 8))
    bars = ax.barh(top_n['sector'].str[:50], top_n['pearson_r'],
                   color=colors, edgecolor='white', height=0.7)

    ax.axvline(0, color='black', linewidth=0.8)
    ax.set_xlabel('Pearson r  (BOE rate vs monthly closures)')
    ax.set_title('Correlation: BOE Rate Rises vs Retail Closure Spikes
by Sector', fontsize=12)
    ax.grid(axis='x', alpha=0.3)

    # Annotate r values
    for bar, r_val in zip(bars, top_n['pearson_r']):
        ax.text(bar.get_width() + 0.01 if r_val >= 0 else
bar.get_width() - 0.01,
                bar.get_y() + bar.get_height() / 2,
                f'{r_val:.2f}',
                va='center', ha='left' if r_val >= 0 else 'right',
fontsize=8)

    plt.tight_layout()
    plt.savefig('phase3_correlation.png', dpi=150)
    plt.show()
    print('✓ Saved: phase3_correlation.png')
```




```
✓ Saved: phase3_correlation.png

# ── 3d. Export correlation results to MySQL ──────────────────────────────
corr_df.to_sql('sector_boe_correlation', engine,
               if_exists='replace', index=False)
print(f'✓ sector_boe_correlation written: {len(corr_df)} rows')

print('\n=== Phase 3 Complete ===')
print('New tables: retail_spend_forecast, sector_boe_correlation')
print('Charts: phase3_forecast.png, phase3_correlation.png')
```

```
✓ sector_boe_correlation written: 44 rows

=== Phase 3 Complete ===
New tables: retail_spend_forecast, sector_boe_correlation
Charts: phase3_forecast.png, phase3_correlation.png
```

Phase 4 — Regional High Street Analysis

This section answers: 1) Which regions lost the most retail businesses (2020–2024)? 2) Which sector groups declined fastest? 3) Does weak spend relate to later closures (lag analysis)?

```
# ── 4a. Build regional + sector decline tables (2020–2024) ──────────────────
import numpy as np
import re

# Basic safety: ensure expected columns exist
required_cols = {'dissolution_date', 'sic_text'}
missing = required_cols - set(closures_raw.columns) if 'closures_raw'
in globals() else required_cols

if missing:
    print(f"Cannot run Phase 4a yet. Missing columns in closures_raw:
{sorted(missing)}")
    region_loss = pd.DataFrame(columns=['region',
'closures_2020_2024'])
    sector_decline = pd.DataFrame(columns=['sector_group',
'closures_2020_2024'])
else:
    tmp = closures_raw.copy()
    tmp['dissolution_date'] = pd.to_datetime(tmp['dissolution_date'],
errors='coerce', dayfirst=True)
    tmp = tmp.dropna(subset=['dissolution_date', 'sic_text']).copy()

    # Keep analysis window
    tmp = tmp[(tmp['dissolution_date'] >= '2020-01-01') &
(tmp['dissolution_date'] <= '2024-12-31')].copy()

    # ---- Improved county/postcode -> region mapping ----
    county_raw = tmp.get('county', pd.Series(index=tmp.index,
dtype='object')).fillna('').astype(str)
    postcode_raw = tmp.get('postcode', pd.Series(index=tmp.index,
dtype='object')).fillna('').astype(str)

    def norm_text(s):
        s = s.upper().strip()
        s = s.replace('&', ' AND ')
        s = re.sub(r'[^A-Z0-9 ]+', ' ', s)
        s = re.sub(r'\s+', ' ', s).strip()
        return s

    def norm_county(s):
        s = norm_text(s)
        for token in ['COUNTY OF', 'COUNTY', 'CITY OF', 'METROPOLITAN
BOROUGH OF', 'LONDON BOROUGH OF']:
            s = s.replace(token, '').strip()
```

```
        return s

    county = county_raw.map(norm_county)
    postcode = postcode_raw.map(norm_text)

    region_keywords = [
        ('London', ['LONDON']),
        ('North East', ['TYNE AND WEAR', 'NORTHUMBERLAND', 'COUNTY
DURHAM', 'DURHAM', 'DARLINGTON', 'SUNDERLAND', 'MIDDLESBROUGH',
'NEWCASTLE']),
        ('North West', ['GREATER MANCHESTER', 'MANCHESTER',
'MERSEYSIDE', 'LANCASHIRE', 'CUMBRIA', 'CHESHIRE']),
        ('Yorkshire and The Humber', ['NORTH YORKSHIRE', 'SOUTH
YORKSHIRE', 'WEST YORKSHIRE', 'EAST RIDING', 'YORKSHIRE', 'KINGSTON
UPON HULL']),
        ('East Midlands', ['NOTTINGHAMSHIRE', 'DERBYSHIRE',
'LEICESTERSHIRE', 'LINCOLNSHIRE', 'NORTHAMPTONSHIRE', 'RUTLAND']),
        ('West Midlands', ['WEST MIDLANDS', 'WARWICKSHIRE',
'WORCESTERSHIRE', 'STAFFORDSHIRE', 'SHROPSHIRE', 'HEREFORDSHIRE']),
        ('East of England', ['NORFOLK', 'SUFFOLK', 'ESSEX',
'CAMBRIDGESHIRE', 'BEDFORDSHIRE', 'HERTFORDSHIRE', 'HERTS',
'PETERBOROUGH', 'LUTON']),
        ('South East', ['KENT', 'SURREY', 'HAMPSHIRE', 'BERKSHIRE',
'OXFORDSHIRE', 'BUCKINGHAMSHIRE', 'EAST SUSSEX', 'WEST SUSSEX', 'ISLE
OF WIGHT']),
        ('South West', ['DEVON', 'CORNWALL', 'SOMERSET', 'DORSET',
'WILTSHIRE', 'GLOUCESTERSHIRE', 'BRISTOL', 'BATH AND NORTH EAST
SOMERSET']),
        ('Wales', ['WALES', 'CARDIFF', 'SWANSEA', 'GWYNEDD',
'WREXHAM', 'NEWPORT', 'POWYS']),
        ('Scotland', ['SCOTLAND', 'LANARKSHIRE', 'MIDLOTHIAN',
'GLASGOW', 'EDINBURGH', 'ABERDEEN', 'DUNDEE', 'HIGHLAND', 'FIFE']),
        ('Northern Ireland', ['NORTHERN IRELAND', 'BELFAST', 'DERRY',
'ANTRIM', 'DOWN', 'TYRONE', 'ARMAGH'])
    ]

    pc_prefix_map = {
        # London
        'E': 'London', 'EC': 'London', 'N': 'London', 'NW': 'London',
'SE': 'London', 'SW': 'London', 'W': 'London', 'WC': 'London',
        # South East
        'GU': 'South East', 'RG': 'South East', 'SL': 'South East',
'OX': 'South East', 'ME': 'South East', 'CT': 'South East', 'TN':
'South East', 'PO': 'South East', 'BN': 'South East',
        # East of England
        'CM': 'East of England', 'CB': 'East of England', 'IP': 'East
of England', 'NR': 'East of England', 'AL': 'East of England', 'LU':
'East of England', 'SG': 'East of England',
        # South West
        'BS': 'South West', 'BA': 'South West', 'TA': 'South West',
```

```
'EX': 'South West', 'PL': 'South West', 'TR': 'South West', 'DT':
'South West', 'SP': 'South West', 'GL': 'South West',
        # West Midlands
        'B': 'West Midlands', 'CV': 'West Midlands', 'DY': 'West
Midlands', 'WR': 'West Midlands', 'WS': 'West Midlands', 'ST': 'West
Midlands',
        # East Midlands
        'DE': 'East Midlands', 'LE': 'East Midlands', 'NG': 'East
Midlands', 'NN': 'East Midlands', 'LN': 'East Midlands',
        # North West
        'M': 'North West', 'L': 'North West', 'PR': 'North West',
'WN': 'North West', 'WA': 'North West', 'CH': 'North West', 'CW':
'North West', 'BB': 'North West', 'LA': 'North West',
        # Yorkshire and The Humber
        'LS': 'Yorkshire and The Humber', 'WF': 'Yorkshire and The
Humber', 'HD': 'Yorkshire and The Humber', 'HU': 'Yorkshire and The
Humber', 'YO': 'Yorkshire and The Humber', 'S': 'Yorkshire and The
Humber', 'DN': 'Yorkshire and The Humber',
        # North East
        'NE': 'North East', 'SR': 'North East', 'DH': 'North East',
'DL': 'North East', 'TS': 'North East',
        # Wales
        'CF': 'Wales', 'SA': 'Wales', 'LL': 'Wales', 'NP': 'Wales',
'SY': 'Wales', 'LD': 'Wales',
        # Scotland
        'EH': 'Scotland', 'G': 'Scotland', 'AB': 'Scotland', 'DD':
'Scotland', 'FK': 'Scotland', 'KY': 'Scotland', 'PH': 'Scotland',
'IV': 'Scotland',
        # Northern Ireland
        'BT': 'Northern Ireland'
    }

    def map_by_county(x):
        if not x:
            return None
        for region, keys in region_keywords:
            if any(k in x for k in keys):
                return region
        return None

    def map_by_postcode(pc):
        if not pc:
            return None
        m = re.match(r'^([A-Z]{1,2})', pc)
        if not m:
            return None
        pref = m.group(1)
        return pc_prefix_map.get(pref)

    region_from_county = county.map(map_by_county)
```

```
    region_from_postcode = postcode.map(map_by_postcode)
    tmp['region'] =
region_from_county.fillna(region_from_postcode).fillna('Other/Unknown'
)

    # Sector grouping from SIC text
    sic = tmp['sic_text'].astype(str).str.lower()
    tmp['sector_group'] = np.select(
        [
            sic.str.contains('clothing|footwear|textile|fashion',
na=False),
            sic.str.contains('food|bakery|grocery|beverage|tobacco|
chemist|pharmacy', na=False),
            sic.str.contains('furniture|electrical|electronics|
hardware|home', na=False),
            sic.str.contains('online|mail order|internet', na=False),
            sic.str.contains('motor|automotive|fuel', na=False),
        ],
        [
            'Fashion',
            'Food & Essentials',
            'Home & Electrical',
            'E-commerce',
            'Automotive/Fuel',
        ],
        default='Other Retail'
    )

    region_loss = (
        tmp.groupby('region').size()
           .reset_index(name='closures_2020_2024')
           .sort_values('closures_2020_2024', ascending=False)
    )

    sector_decline = (
        tmp.groupby('sector_group').size()
           .reset_index(name='closures_2020_2024')
           .sort_values('closures_2020_2024', ascending=False)
    )

    unknown_n = int((tmp['region'] == 'Other/Unknown').sum())
    unknown_pct = (unknown_n / len(tmp) * 100) if len(tmp) else 0
    top_unknown = county[tmp['region'] ==
'Other/Unknown'].value_counts().head(10)

    print('region_loss rows:', len(region_loss))
    print(region_loss.head(12).to_string(index=False))
    print(f"\nOther/Unknown rows: {unknown_n} ({unknown_pct:.1f}%)")
    if not top_unknown.empty:
        print('\nTop unmapped county values:')
```

```
        print(top_unknown.to_string())

    print('\nsector_decline rows:', len(sector_decline))
    print(sector_decline.to_string(index=False))
```

```
region_loss rows: 13
                  region  closures_2020_2024
                  London                5025
           Other/Unknown                4918
              North West                2922
           West Midlands                2496
                   Wales                2271
              South East                1916
         East of England                1763
Yorkshire and The Humber                1628
           East Midlands                1398
                Scotland                1263
              South West                1055
              North East                 642

Other/Unknown rows: 4918 (17.7%)

Top unmapped county values:
county
                4391
MIDDLESEX        236
FAILSWORTH        71
AYRSHIRE          21
ENGLAND           17
BOLTON            13
BUCKS             13
MIDDX             12
RENFREWSHIRE      10
BRADFORD           9

sector_decline rows: 6
     sector_group  closures_2020_2024
     Other Retail               11752
       E-commerce                7134
Food & Essentials                4725
          Fashion                2686
Home & Electrical                1383
  Automotive/Fuel                  69
```

```
# ── 4b. Lag analysis: spend today vs closures later (0–12 months)
──────────────
if {'ds', 'y'}.issubset(spend.columns) and {'ds',
'total_closures'}.issubset(closures_monthly.columns):
    lag_base = (
        spend[['ds', 'y']].dropna().groupby('ds', as_index=False)
```

```
['y'].mean()
        .merge(closures_monthly[['ds', 'total_closures']], on='ds',
how='inner')
        .sort_values('ds')
    )

    lag_results = []
    for lag in range(0, 13):
        tmp = lag_base.copy()
        tmp['closures_lagged'] = tmp['total_closures'].shift(-lag)  #
closures lag months after spend
        valid = tmp.dropna(subset=['y', 'closures_lagged'])
        if len(valid) < 12 or valid['closures_lagged'].std() == 0 or
valid['y'].std() == 0:
            corr = np.nan
        else:
            corr = valid['y'].corr(valid['closures_lagged'])
        lag_results.append({'lag_months': lag, 'pearson_r': corr,
'n_obs': len(valid)})

    lag_df = pd.DataFrame(lag_results)
    best_row = lag_df.iloc[lag_df['pearson_r'].abs().idxmax()] if
lag_df['pearson_r'].notna().any() else None

    print(lag_df.to_string(index=False))
    if best_row is not None:
        print(f"\nStrongest relationship at
lag={int(best_row['lag_months'])} months,
r={best_row['pearson_r']:.3f}")
    else:
        print('\nInsufficient variation/data for lag correlation.')
else:
    lag_df = pd.DataFrame(columns=['lag_months', 'pearson_r',
'n_obs'])
    print('Cannot run lag analysis. Required columns missing in spend
or closures_monthly.')
```

lag_months	pearson_r	n_obs
0	0.315240	177
1	0.314037	176
2	0.308565	175
3	0.302068	174
4	0.284576	173
5	0.271816	172
6	0.259775	171
7	0.259858	170
8	0.272461	169
9	0.275115	168
10	0.254478	167
11	0.248639	166

```
         12   0.237944    165

Strongest relationship at lag=0 months, r=0.315
```

```
# ── 4c. Export Phase 4 outputs to MySQL ──────────────────────────────
region_loss.to_sql('phase4_region_losses', engine,
if_exists='replace', index=False)
sector_decline.to_sql('phase4_sector_decline', engine,
if_exists='replace', index=False)
lag_df.to_sql('phase4_spend_closure_lags', engine,
if_exists='replace', index=False)

print(f"✓ phase4_region_losses written: {len(region_loss)} rows")
print(f"✓ phase4_sector_decline written: {len(sector_decline)} rows")
print(f"✓ phase4_spend_closure_lags written: {len(lag_df)} rows")
```

```
✓ phase4_region_losses written: 13 rows
✓ phase4_sector_decline written: 6 rows
✓ phase4_spend_closure_lags written: 13 rows
```

```
# ── DIAG: inspect raw dissolutions date/status formats ──────────────
diag =
pd.read_csv('~/Desktop/uk-high-street/data/dissolutions_YYYY_MM.csv',
nrows=200000, low_memory=False)
print('Rows sampled:', len(diag))
print('DissolutionDate dtype:', diag['DissolutionDate'].dtype)
print('Non-null DissolutionDate:',
diag['DissolutionDate'].notna().sum())
print('Top CompanyStatus values:\n',
diag['CompanyStatus'].value_counts(dropna=False).head(15))
print('\nDissolutionDate sample raw values:')
print(diag['DissolutionDate'].dropna().head(20).tolist())
```

```
Rows sampled: 200000
DissolutionDate dtype: float64
Non-null DissolutionDate: 0
Top CompanyStatus values:
 CompanyStatus
Active                                                  183683
Active - Proposal to Strike off                          12435
Liquidation                                               3617
In Administration                                          147
Live but Receiver Manager on at least one charge            81
Voluntary Arrangement                                       16
In Administration/Administrative Receiver                   12
RECEIVERSHIP                                                 5
ADMINISTRATION ORDER                                         2
VOLUNTARY ARRANGEMENT / RECEIVER MANAGER                     1
```

```
In Administration/Receiver Manager                          1
Name: count, dtype: int64

DissolutionDate sample raw values:
[]
```